

Milestone Three Enhancement Two: Algorithms & Data Structures Enhancement

Zain Cronk

Southern New Hampshire University

CS-499 Computer Science Capstone

Instructor Jeff Sanford

September 22, 2026

The artifact I selected for the algorithms and data structures category is the Android Inventory Application that I originally developed in CS 360: Mobile Architecture and Programming. The application was developed in Java using Android Studio and uses SQLite to store inventory records. Before this enhancement, the application allowed users to add, edit, delete, and adjust the quantities of inventory items. I selected this artifact because its existing use of an ArrayList to manage inventory records provided an opportunity to improve how users locate and organize data while demonstrating the application of algorithms within an existing software project.

For this enhancement, I added search and sorting functionality to the inventory screen. Users can now search inventory records by entering part of an item's name, with the application performing a case-insensitive linear search through the inventory collection and displaying only matching records. I also added sorting options that allow inventory records to be ordered alphabetically in ascending or descending order or numerically by quantity from lowest to highest or highest to lowest. The search and sorting functionality was designed to work together, allowing a filtered set of inventory records to remain filtered while a selected sorting operation is applied. This improved the usability of the application while demonstrating how algorithms can be applied to an existing data structure to solve practical problems.

The enhancement supports Course Outcomes 3 and 4 as planned. It demonstrates Course Outcome 3 through the design and implementation of algorithms for searching and sorting inventory data stored in an ArrayList. The search operation examines the inventory records sequentially, resulting in $O(n)$ time complexity as the number of records increases. Sorting uses Java's collection and comparator functionality to order the data by different attributes while avoiding unnecessary duplication of sorting logic. The enhancement demonstrates Course

Outcome 4 through the use of Java, Android development tools, collection classes, comparators, RecyclerView, and event-driven interface controls to provide functionality that adds value to the application.

One of the most important parts of the enhancement was determining how search and sorting should interact. Implementing each feature independently was straightforward, but initially applying a sort after a search would have restored the complete inventory list instead of sorting only the filtered results. I addressed this by separating the sorting logic into a reusable method and applying the selected sort operation to the filtered collection before displaying it. This reinforced the importance of considering how algorithms interact with application state rather than evaluating individual features in isolation. I also added four automated instrumented tests that verify search filtering, alphabetical sorting, quantity sorting, and combined search-and-sort behavior. The complete instrumented test suite passes all nine tests, including the tests from the previous enhancement. This testing helped confirm that the new algorithms behave as expected without introducing regressions into the existing application.